

GPT-6 Sol across every effort level

VulcanBench Frontier v4 | Code quality protocol v3.17 | September 2026

Abstract

GPT-6 Sol ran the 23-task VulcanBench Frontier v4 suite through Codex CLI 0.155.0 on a ChatGPT Pro subscription on September 28 to 29, 2026, once per task at each of five effort levels, Low to Max, 115 runs in all. Code quality carries 33% of the combined score and is judged for a named human reader by Muse Spark 1.3 (Meta) and Grok 4.6 (xAI) under the same frozen protocol as the other Frontier v4 reports, with a ground-truth intent-recovery probe. The combined score rises at every step, from 67.62 at Low to 80.60 at Medium, 82.83 at High, 85.94 at Extra-high and 86.82 at Max. GPT-6 Sol passes 4, 13, 15, 18 and 19 of 23 tasks from Low to Max. Code quality rises from 62.53 to 70.78. Cost per task runs from \$1.02 at Low to \$2.52 at Max; the whole sweep prices at \$221.38, \$1.93 per task.

Every run finished inside the flat 3-hour task bound, so nothing is excluded. One Medium run, codeccore, has no published Code quality score: Grok 4.6's intent probe gave no valid answer after the protocol's single retry, so the Medium cell's Code quality and combined score cover 22 of its 23 runs. The run failed its tests and is counted in every pass count, runtime, token and cost figure. Beside GPT-5.6 Sol, GPT-6 Sol scores slightly lower at every level, is slower, and costs more per task from Medium up.

Table 1. Combined score, Code quality, runtime and tasks passed by effort

Effort	Combined (33%)	SE	Combined (20%)	Code quality	Min/task	Judged	Passed
Low	67.62	3.59	71.23	62.53	13.0	23	4/23
Medium	80.60	2.59	83.77	66.79	17.5	22	13/23
High	82.83	2.43	85.46	70.09	23.8	23	15/23
Extra-high	85.94	1.38	88.38	70.77	20.9	23	18/23
Max	86.82	0.91	89.19	70.78	24.5	23	19/23

Combined (33%) = 0.50 functional + 0.085 lint and complexity + 0.085 security + 0.33 Code quality, over judged runs. Combined (20%) applies the prior 50/15/15/20 profile to the same Code quality scores. SE is one sample standard error across the cell's judged tasks, not judge uncertainty or a significance test. Passed counts all 23 runs. Runtime is solver wall-clock per task over every run in the cell; judging is excluded.

The Code quality protocol

Two failure modes motivated the reviewed score. The lint and complexity metric rewards compression: the maintainability index carries a lines-of-code term and per-function complexity stays low when each dense line does something different, so five statements on a line with names like *x* and *k* can outscore the same logic written for a person. And a large model reading compressed code pays almost nothing to parse it; when a frontier model was calibrated as a readability judge, it rated a squashed six-line function the same as its formatted copy. Since September 7, 2026 Code quality carries 33% of the combined score and lint and complexity and security carry 8.5% each. The prior profile is reported beside the new one on every table.

Protocol v3.17 is the same protocol applied to this population. Nothing in the rubric, controls, quirk keys, gates, repeats, seed, weights or judges changed from v3.7; both judges retook the calibration exam under v3.17 before any counted call. The population froze on September 29 with 115 rows, none missing and none excluded. A submission without a valid answer-key match from every passing panel is left unpublished by the frozen summary; that rule applied once, described under Results in detail.

The rubric

Every judge receives the same instructions, frozen by hash before any review. The prompt names the reader it scores for: an engineer who has never seen the code, reads it top to bottom without running it, and must make a correct change in one sitting. It tells the judge that its own ease at parsing dense code is not evidence of readability. Six dimensions are scored 0 to 4 in half steps against written anchors, each with an exact excerpt and a concrete consequence for that reader.

Sub-score	Dimension	What is assessed
Human readability	Naming	Identifiers say what things are in the task's domain
Human readability	Presentation	Statements per line, nesting, function length, how much the reader must hold at once
Human readability	Intent	Constants, thresholds and quirks explained by names or accurate comments that say why
Maintainability	Structure	Coherent responsibilities, no duplicated policy, no abstraction beyond the task's scale
Maintainability	Changeability	A named plausible change, every edit site traced, scored on locality and safety
Maintainability	Verifiability	Explicit state, failures that name what went wrong, seems to test one rule alone

The layers under the 33 points

Reviewed panel, 24 points. The six-dimension rubric above, averaged across both judges. **Intent recovery, 9 points.** Every task in this suite is a rewrite of a retired binary whose real behaviour departs from its written specification in documented ways. The judge sees only the specification and the code, never the issue text, and lists where the code departs from the specification; a separate call matches that list against the frozen answer key. The denominator is the quirks the submission actually passed tests for, so a functional failure is not punished twice; a submission that passed none would have no intent-recovery score and its Code quality would be the reviewed score alone, which no GPT-6 Sol run needed. **Measured maintenance, designed for 12 points, not yet built.** Until it exists the pre-registered split above is in force.

Judges and calibration

The scored panel is Muse Spark 1.3 through the Muse CLI on its Standard tier, which does not train on prompts, and Grok 4.6 through the Cursor CLI at medium effort, the v3.7 pair under the same pinned binaries. Neither lab has a model on this board, so neither judge grades a relative. Every session is fresh, tools are disabled, the workspace is empty, model identity is checked per call from the CLI's own records, and solver labels are withheld.

The calibration exam

Before scoring a single submission each judge reviews ten held-out programs that implement the same ledger specification: clear, compressed, compressed then auto-formatted, verbose with duplicated policy, needlessly abstracted, misleadingly commented, narrated with a comment on every line, a documented legacy quirk, an embedded instruction to give full marks, and hidden module state. Each program is reviewed five times in a seeded order. Twenty gates fixed in advance check that the judge sees the construct; a judge may miss at most one gate by at most half a point.

Table 2. Calibration verdicts

Judge	Protocol	Calls	Result	Allowance	Failing gates
Muse Spark 1.3	code-quality-maintenance-v3.17	80	passed	not used	none
Grok 4.6	code-quality-maintenance-v3.17	80	passed	not used	none

Both judges passed every gate with no allowance used; under v3.16 each had needed the pre-registered one-gate allowance.

Table 3. Control means on five of the ten calibration programs

Judge	Control	Naming	Present.	Intent	Struct.	Change.	Verif.
Muse	clear	4.00	3.80	3.50	4.00	3.90	3.60
Muse	compressed	1.00	1.70	1.10	3.70	2.70	3.00
Muse	formatted	1.30	3.20	1.20	3.90	3.00	3.10
Muse	misleading comments	3.90	3.80	0.10	4.00	2.40	3.40
Muse	hidden state	2.50	2.90	2.10	1.70	2.60	1.90
Grok	clear	3.60	3.50	3.20	4.00	4.00	3.50
Grok	compressed	1.50	1.50	1.40	3.10	2.60	3.00
Grok	formatted	1.50	2.40	1.50	3.40	3.10	3.00
Grok	misleading comments	3.50	3.10	1.10	3.40	2.40	3.00
Grok	hidden state	2.90	2.90	2.50	2.00	3.30	2.30

Full gate values and control means are in calibration.json.

Results in detail

Table 4. Code quality components at each effort level

Component	Low	Medium	High	Extra-high	Max
Code quality	62.53	66.79	70.09	70.77	70.78
Human readability	53.8	60.0	62.1	61.7	63.4
Maintainability	65.5	68.5	72.5	72.7	73.6
Intent recovery	70.2	73.6	77.5	80.3	76.9
Reviewed score	59.6	64.3	67.3	67.2	68.5
Rated by Muse Spark 1.3	57.9	63.1	66.6	66.7	68.2
Rated by Grok 4.6	61.4	65.4	68.0	67.8	68.8
Standard error of Code quality	1.61	1.49	1.57	1.34	1.40
Judged runs	23	22	23	23	23

Code quality rises from 62.53 at Low to about 70.8 at Extra-high and Max, with Grok 4.6 the higher rater at every level. Means cover judged runs.

Table 5. Four factors by effort, mean score out of 100 over judged runs

Effort	Functional	Lint, complexity	Security	Code quality
Low	63.27	80.60	100.00	62.53
Medium	86.13	83.21	99.09	66.79
High	88.67	84.44	96.30	70.09
Extra-high	94.72	83.23	95.87	70.77
Max	96.64	83.99	94.13	70.78

The unpublished Medium row

On codecore at Medium, Grok 4.6's intent probe produced no valid answer after the protocol's single retry. Attempt 1 was not valid JSON; attempt 2 quoted `memo = record[31:46].rstrip(". ").replace(" ", "")` if False else `record[31:46].rstrip(". ")`, which is not in the code. No recovery rule accepts an invented excerpt, so no answer-key match was made and the frozen summary publishes no Code quality or combined score for the run. It is the same codecore line Grok 4.6 misquoted for GPT-5.6 Sol at Max under v3.7. Both judges' reviews and the Muse probe are retained in the harness archive. The run failed its tests, so it counts as a failed task, and its runtime, tokens and cost are in every economics figure. The operator's finding is recorded beside the two attempts and its hash is in provenance.json.

Operator record

On high payrollcore, both of Grok 4.6's primary-review attempts returned a complete JSON review followed by a stray closing brace. The formatting-only rule `recover_trailing_braces` decoded the first JSON object, dropped the trailing brace and applied every normal check; attempt 1 passed and was selected, reviewed score 79.17. No field was edited. Under v3.17 Muse Spark 1.3 made 440 counted calls (80 in calibration, 115 primary reviews, 5 repeats, 10 pairwise checks, 115 intent probes, 115 answer-key matches) and Grok 4.6 made 439, with 114 matches. Muse needed a second attempt on six calls, all unsupported excerpts; Grok on five, three primary reviews with a trailing brace and two probes. Neither judge produced a reviewer fallback.

Cost and tokens across effort levels

All 115 runs priced from their Codex receipts at list API rates, cache-aware, solver inference only, judging excluded. GPT-6 Sol ran on a ChatGPT Pro subscription, so these are API-equivalent estimates rather than bills. Low is the cheapest and fastest level; High uses more tokens and costs more per task than Extra-high, and Max is the most expensive. Every run is priced, including the Medium run without a published Code quality score.

Table 6. API-equivalent cost, raw tokens and runtime by effort

Effort	Runs	\$/task	Level total	Tokens/task	Min/task
Low	23	\$1.02	\$23.45	3.26M	13.0
Medium	23	\$1.76	\$40.44	6.16M	17.5
High	23	\$2.33	\$53.60	8.28M	23.8
Extra-high	23	\$2.00	\$45.91	6.68M	20.9
Max	23	\$2.52	\$57.97	8.32M	24.5
Full sweep	115	\$1.93	\$221.38	752M	38.2 h

Rates checked 2026-09-25: GPT-6 Sol at \$2.00 input, \$0.20 cached input and \$10.00 output per million tokens. Codex receipts report input, cached input, output and reasoning tokens per run, so cached input is billed at the cache-read rate and the rest at the standard rate. The sweep stamped each run at these rates at run time; the export recomputed every run from its receipt and matched every stamp. Tokens are raw solver totals including cache reads.

Limitations recorded with the estimates

- Codex receipts report input, cached input, output and reasoning tokens per run; cached input is billed at the cache-read rate and the rest of the input at the standard rate. Cache writes are free on this API and are not modelled.
- Standard tier, short-context rates. OpenAI bills prompts over 272K input tokens at a higher rate, but Codex keeps each request within its 272K context window, so no long-context premium applies and none is modelled.
- No Batch, Flex, Fast or priority pricing is applied.
- The estimate covers the solver's exposed receipts, not an independently observed API invoice.
- The sweep stamped each run at run time at the published GPT-6 Sol list rates; the export recomputes every run from its receipt and matched every stamp.

Beside GPT-5.6 Sol

GPT-5.6 Sol ran the same 23 tasks through Codex in September and was judged by the same judges under v3.7 (Max judged on 22 of 23). GPT-6 Sol's combined score is slightly lower at every level; the gaps at Extra-high and Max are within one standard error of either model. At half the list price per token, GPT-6 Sol uses 1.8 to 3.9 times the raw tokens per task, so it costs less only at Low.

Table 7. GPT-6 Sol and GPT-5.6 Sol by effort

Effort	GPT-6 Sol	GPT-5.6 Sol	Passed of 23	\$/task	Min/task
Low	67.62	69.05	4 / 6	\$1.02 / \$1.36	13.0 / 9.8
Medium	80.60	82.56	13 / 12	\$1.76 / \$1.72	17.5 / 11.1
High	82.83	85.76	15 / 20	\$2.33 / \$2.05	23.8 / 12.0
Extra-high	85.94	86.47	18 / 21	\$2.00 / \$1.61	20.9 / 10.6
Max	86.82	87.18	19 / 22	\$2.52 / \$1.77	24.5 / 11.6

Combined scores are over judged runs; pass counts, cost and minutes cover all 23 runs per cell, GPT-6 Sol first in each pair. Each model is priced at its own list rates (GPT-5.6 Sol \$4.00 input and \$20.00 output per million tokens). The sweeps ran on different Codex CLI versions (0.155.0 and 0.153.4).

How it was run

Codex CLI 0.155.0. The earlier Codex columns ran on 0.153.4, which refuses GPT-6 models on a ChatGPT account before any work; 0.155.0 is the lowest release that serves them and was installed beside the global CLI for the GPT-6 sweeps only. **The sweep overlapped another model's judging.** The solver sweep ran September 28, 00:31 PDT, to September 29, 14:53 PDT; at the owner's request it ran alongside GPT-6 Luna's judging, September 28, 00:32 to 14:32 PDT, which matters for the wall-clock figures. **One infrastructure retry.** The first Max depotcore attempt ended after eight minutes when the API reported the model at capacity; the harness re-queued it and the retry is the counted run. **Judging** ran September 29, 14:54 PDT, to September 30, 06:51 PDT, resumed after the two Grok findings on page 4, while GPT-6.1 Sol's solver sweep ran at the owner's request; the judges share no quota with it. **One attempt per task and level;** effort labels are Codex's own.

Evidence and reproduction

Public files: `runs.json` and `runs.csv` with every run's factors, both judges' six-dimension sub-scores and intent recovery for each published run; `groups.json`; `calibration.json` with every gate value and control mean; `judge-protocols.json` with the exact rubric, system text, probe and match instructions, schemas, weights, the finding on the unpublished row and the one operator recovery; `economics.json` with cost and token aggregates, the rate table and pricing limitations; and `provenance.json` with source hashes and limits.

<https://github.com/morganlinton/VulcanBenchCOM/tree/main/assets/data/swe-v4-gpt6-sol-v317>

Withheld: raw prompts and responses; submitted patches and reconstructed sources; quirk answer keys (they describe hidden-test behaviour); reviewer session identifiers and usage receipts.

Recompute the published numbers

Each run's Code quality is $(0.24 \times \text{reviewed score} + 0.09 \times \text{intent recovery}) / 0.33$, where both terms are the equal mean of the two judges, or the reviewed score alone where the submission passed no quirk family. The combined score is $100 \times (0.50 F + 0.085 Q + 0.085 S + 0.33 C / 100)$ with F, Q, S on a 0 to 1 scale. Group means weight the cell's judged tasks equally. The export script recomputes every row from the frozen summary, re-prices every run from its receipt, asserts that exactly one row is unpublished and that it is the row carrying the operator's invalid-probe marker, checks the figures against the harness card tables, and refuses to write if any value differs.

What this report does not claim

No human rated anything; the scores are model judgment for a human reader, not human validation. Standard errors describe task sampling only. The measured-maintenance layer is unbuilt, so the 33 points are 24 reviewed plus 9 intent recovery. Runs were not repeated. The Medium cell's Code quality and combined score are means over 22 of 23 runs. Hash checks bind the export to frozen files; they do not prove that judges were unbiased or that no training overlap exists.

Source hashes

Frozen artifact	SHA-256
v3.17/summary.json	83e6e9d1c5dffffaaae0e5b2388462f0a...
v3.17/protocol.json	6d73cb7d4320e20114ef70c73a6a250c...
v3.17/private-manifest.json	50ef3caa26840fc6fd2f794ee1533840...
v3.17/calibration-muse.json	17b42cc36c50b77823473ebe39545075...
v3.17/calibration-grok.json	7ce053e62665e8914dec9d7ebba1ed9c...
calls/grok/probe/submission-061/operator-invalid.json	7657745ce5c34175eb3a32dc204f7af0...
calls/grok/primary/submission-035/selected.json	d7fb747a4ea11798b54f9a1f1bf33c83...
comparison.json	065f80a4dd750ecf8144f06659d71e1f...
gpt6-sol-v317-efforts.csv	cf648e7498f5364dbc873f49ddc0288e...
gpt6-sol-v317-economics-efforts.csv	2039fba71673d0e5323e11df94b5612e...
gpt6-vs-gpt56-sol-efforts.csv	0e3c0f189bf26b079c79196ff57618b9...
gpt6-vs-gpt56-all-efforts.csv	f155eb49538f37d4c7dcfec71cf036cf...

Appendix. Per task at Low, Medium and Max

Task	Low CQ	Max CQ	Low combined	Medium combined	Max combined
blendcore	57.2	68.5	67.73	88.87	87.86
cellarcore	54.7	63.4	32.98	86.57	85.48
codeccore	63.1	73.6	56.14	unpublished	74.53
depotcore	51.5	64.0	71.15	78.55	86.42
freightcore	69.9	77.3	38.86	40.27	90.40
granarycore	49.5	62.1	55.72	69.69	84.66
hedgcore	54.0	74.5	77.04	88.52	90.19
lodgcore	56.7	68.9	80.00	89.53	87.97
matchcore	73.5	72.0	39.40	77.49	89.39
metercore	69.9	71.4	88.46	87.88	88.38
pacecore	75.8	81.1	40.81	89.70	86.91
paddockcore	50.0	50.0	63.88	70.55	81.28
payrollcore	64.3	73.8	76.66	86.42	89.09
qlite	69.1	79.9	82.55	89.56	85.56
queuecore	69.4	68.6	69.98	73.52	75.38
quotacore	67.2	75.0	78.09	88.61	90.61
reflowcore	61.7	64.8	73.76	86.18	86.19
schedcore	60.7	76.9	85.11	90.36	90.07
settlecore	63.3	71.2	75.95	73.21	87.73
snapcore	56.8	72.3	49.26	63.30	89.75
stampcore	67.1	70.9	87.62	88.26	89.12
tallycore	60.8	73.5	85.90	89.37	90.02
vaultcore	72.0	74.2	78.22	76.79	89.83

Per-task values from runs.json. Task names are shortened for width. The codeccore Medium run is unpublished for the reason given on page 4.

VulcanBench Frontier v4: GPT-6 Sol across effort levels

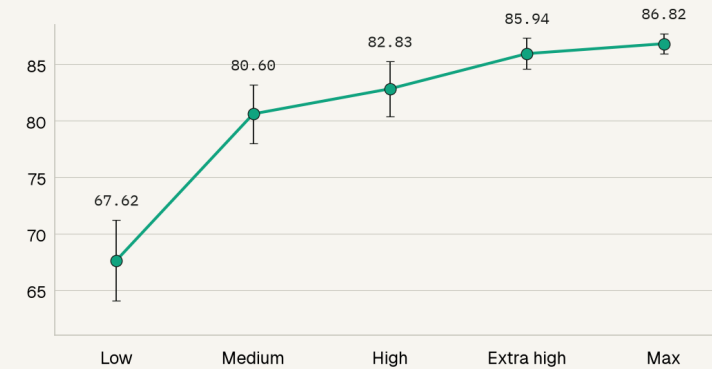
Combined score and runtime at every effort level, 23 tasks per effort. Code quality judged by Muse Spark 1.3 and Grok 4.6.

● **GPT-6 Sol** · **Codex 0.155.0**

n=23 at every effort, n=22 judged at medium

Combined score

/100 · higher is better · focused scale



Mean runtime

Minutes per task · lower is better

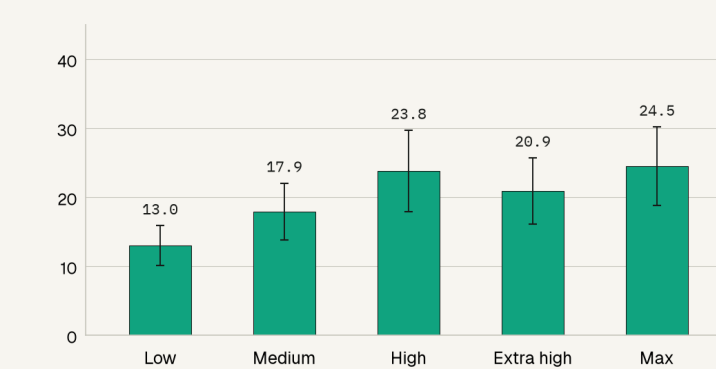


Table 1 | Code quality at each effort level

Component	Low	Medium	High	Extra high	Max
/100, mean of both judges					
Code quality	62.53	66.79	70.09	70.77	70.78
Human readability	53.8	60.0	62.1	61.7	63.4
Maintainability	65.5	68.5	72.5	72.7	73.6
Intent recovery	70.2	73.6	77.5	80.3	76.9
Rated by Muse Spark 1.3	57.9	63.1	66.6	66.7	68.2
Rated by Grok 4.6	61.4	65.4	68.0	67.8	68.8
Standard error of Code quality	1.61	1.49	1.57	1.34	1.40

All 115 runs finished inside the flat 3-hour task bound (longest: max paddockcore, 129 minutes); none is excluded as a timeout.

Ran on Codex CLI 0.155.0, the first release that serves GPT-6 Sol on a ChatGPT plan; earlier Codex columns used 0.153.4 or older.

Medium is judged on 22 of 23 runs: on codecore, Grok 4.6 gave no valid intent probe (one attempt quoted code absent from the run, the other was malformed), so the protocol publishes no Code quality score for that run. It counts for tasks passed, time and cost, as GPT-5.6 Sol's max codecore run did under v3.7.

VulcanBench Frontier v4: GPT-6 Sol across effort levels

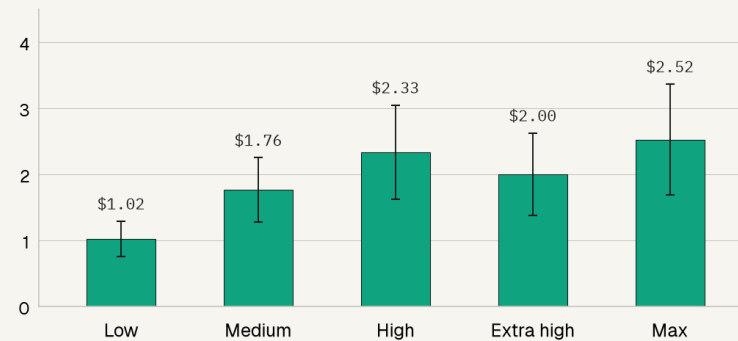
API-equivalent cost and token use at every effort level. All 115 runs of the sweep; solver inference only, judging excluded.

● **GPT-6 Sol · Codex 0.155.0**

n=23 at every effort

API-equivalent cost

USD per task at list prices · lower is better



Tokens per task

Millions of raw tokens, cache reads included · lower is better

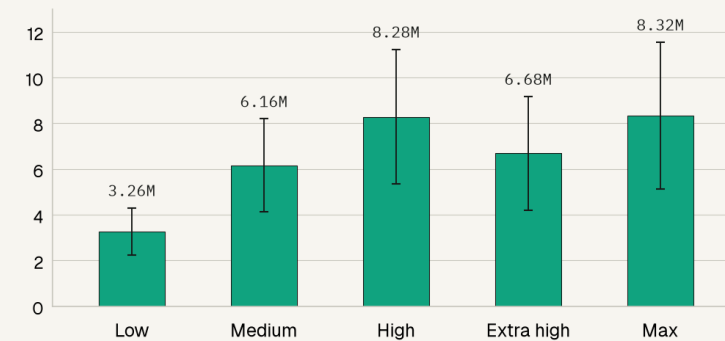


Table 1 | Cost, tokens and runtime at each effort level

Effort	Runs	\$/task	Level total	Tokens/task	Minutes/task
			API-equivalent, list prices	raw, millions	solver wall-clock
Low	23	\$1.02	\$23.45	3.26M	13.0
Medium	23	\$1.76	\$40.44	6.16M	17.5
High	23	\$2.33	\$53.60	8.28M	23.8
Extra high	23	\$2.00	\$45.91	6.68M	20.9
Max	23	\$2.52	\$57.97	8.32M	24.5
Full sweep	115	\$1.93	\$221.38	752M	38.2 h

USD at list prices checked 2026-09-25, cache-aware (GPT-6 Sol \$2.00 input, \$0.20 cached input, \$10.00 output per million), solver inference only; subscription bills differ.

All 115 runs are priced; none reached the 3-hour task bound.

Whiskers are one task standard error. Tokens are raw solver totals including cache reads.